Agent Skills

+ OpenCode Setup Guide

Production-Grade Engineering Skills for AI Coding Agents

Step-by-Step Installation and Configuration

Based on addyosmani/agent-skills (MIT License)

Guide generated: August 2026

1. Overview

Agent Skills is a collection of 24 structured workflows that enforce senior engineering best practices across the software development lifecycle. When used with OpenCode, skills are automatically selected based on your intent - no manual commands needed.

What You Get

- 24 skills covering Define, Plan, Build, Verify, Review, and Ship phases
- Automatic skill activation based on what you are doing
- Anti-rationalization tables that prevent skipping quality steps
- Verification gates that require evidence before moving on
- Works with OpenCode free AI model (no paid API required)

Requirements

- OpenCode installed and configured
- Git installed
- A terminal / command prompt
- Internet connection (for initial clone only)

2. Installation

Option A: Clone for a Specific Project

Best for: You want skills in one project only.

1. Open your terminal and navigate to your project folder.

```
cd D:\MyProjects\my-new-app
```

2. Clone the agent-skills repo into a temp location.

```
git clone https://github.com/addyosmani/agent-skills.git C:\temp\agent-skills
```

3. Copy the two required items into your project.

```
copy C:\temp\agent-skills\AGENTS.md .
xcopy /E /I C:\temp\agent-skills\skills .\skills
```

4. Verify the files exist.

```
dir AGENTS.md
Dir skills\
```

5. Open your project in OpenCode.

```
opencode .
```

Option B: Global Install (All Projects)

Best for: You want skills available everywhere.

1. Clone once to a permanent location.

```
git clone https://github.com/addyosmani/agent-skills.git C:\agent-skills
```

2. Create a global OpenCode config if you do not have one.

```
mkdir %USERPROFILE%\.config\opencode
```

3. Create or edit the config file at %USERPROFILE%\.config\opencode\config.json

```
{
  "skills": "C:\\agent-skills\\skills"
}
```

4. Copy AGENTS.md to each project that needs skills, or add it to your global instructions.

3. Your Project Structure

After installation, your project should look like this:

```
my-new-app/
  AGENTS.md              <-- OpenCode reads this
  skills/                <-- 24 skill workflows
    spec-driven-development/
      SKILL.md
    planning-and-task-breakdown/
      SKILL.md
    incremental-implementation/
      SKILL.md
    test-driven-development/
      SKILL.md
    code-review-and-quality/
      SKILL.md
    ... (20 more skills)
  src/                   <-- Your source code
  package.json           <-- Your project files
  ...
```

The agent reads AGENTS.md on startup and automatically knows which skills to invoke based on your requests.

4. How Skills Activate

You do NOT need to manually invoke skills. The agent automatically maps your intent to the right skill. Here is the mapping:

Intent to Skill Mapping

- "Add a login page" -> spec-driven-development -> incremental-implementation
- "Plan the refactor" -> planning-and-task-breakdown
- "Fix this bug" -> debugging-and-error-recovery
- "Review my code" -> code-review-and-quality
- "Design the API" -> api-and-interface-design
- "Build the UI" -> frontend-ui-engineering
- "Write tests" -> test-driven-development
- "Ship to production" -> shipping-and-launch

Development Lifecycle

The agent follows this lifecycle internally:

- DEFINE: spec-driven-development
- PLAN: planning-and-task-breakdown
- BUILD: incremental-implementation + test-driven-development
- VERIFY: debugging-and-error-recovery
- REVIEW: code-review-and-quality
- SHIP: shipping-and-launch

5. Key Prompts to Use

Copy-paste these prompts into OpenCode. The agent will automatically select the right skill and follow the workflow.

PROMPT 1: Start a New Feature with Spec

```
I want to add user authentication to my app.
Follow the spec-driven-development skill.
Ask me questions one at a time until you have
enough information to write a complete PRD.
```

PROMPT 2: Plan Before Building

```
I have a spec for a new checkout feature.
Use the planning-and-task-breakdown skill to
decompose it into small, verifiable tasks with
acceptance criteria and dependency ordering.
```

PROMPT 3: Build with TDD

```
Implement the password reset feature.
Use test-driven-development: write failing tests
first, make them pass, then refactor.
Follow the red-green-refactor cycle strictly.
```

PROMPT 4: Incremental Implementation

```
Build the shopping cart feature one thin vertical
slice at a time. Use incremental-implementation:
implement, test, verify, commit for each slice.
Use feature flags for safe rollout.
```

PROMPT 5: Code Review

```
Review the changes in the last commit.
Use code-review-and-quality with a five-axis
review: correctness, design, readability,
test coverage, and security.
```

PROMPT 6: Debug a Failing Test

```
My test "test_login_redirect" is failing with
a 401 error. Use debugging-and-error-recovery:
reproduce, localize, reduce, fix, guard.
```

PROMPT 7: Security Audit

```
Audit my authentication module for security
vulnerabilities. Use security-and-hardening.
Check OWASP Top 10, input validation, and
secrets management.
```

PROMPT 8: Ship to Production

```
I am ready to deploy. Use shipping-and-launch.
Run the pre-launch checklist, verify feature
flags, confirm rollback plan, and set up
monitoring.
```

PROMPT 9: Auto-Build (Full Lifecycle)

```
I have a complete spec. Use /build auto to
generate the plan and implement every task
autonomously. Pause only on failures or
risky steps. Commit each task individually.
```

PROMPT 10: Simplify Existing Code

```
The src/utils/parser.ts file is hard to read.
Use code-simplification: apply Chesterton's
Fence, reduce complexity while preserving
exact behavior, and improve readability.
```

6. Advanced: Custom System Prompt

For best results with free AI models, add this to your AGENTS.md or as a system prompt prefix. It reinforces the skill workflow:

SYSTEM PROMPT (prepend to AGENTS.md)

```
You are a senior software engineer following
production-grade engineering practices.

RULES:
1. Before writing any code, check if a skill
   applies. Skills are in skills/<name>/SKILL.md
2. If a skill applies, you MUST follow it.
3. Never skip: spec, plan, test, review steps.
4. Every change must have passing tests.
5. Every commit must be atomic (~100 lines).
6. Verify everything with evidence.
7. "Seems right" is never sufficient.

ANTI-RATIONALIZATION:
- Do NOT say "this is too small for a skill"
- Do NOT say "I will add tests later"
- Do NOT skip the spec phase
- Always use the correct skill workflow
```

7. Skills Quick Reference

Define Phase

- interview-me: One-question-at-a-time interview to extract real requirements
- idea-refine: Structured divergent/convergent thinking for rough ideas
- spec-driven-development: Write a PRD before any code

Plan Phase

- planning-and-task-breakdown: Decompose specs into verifiable tasks

Build Phase

- incremental-implementation: Thin vertical slices, test-driven
- test-driven-development: Red-Green-Refactor, test pyramid
- context-engineering: Feed agents the right info at the right time
- source-driven-development: Ground decisions in official docs
- doubt-driven-development: Adversarial review of non-trivial decisions
- frontend-ui-engineering: Components, design systems, accessibility
- api-and-interface-design: Contract-first, Hyrum's Law

Verify Phase

- browser-testing-with-devtools: Chrome DevTools for live debugging
- debugging-and-error-recovery: Five-step triage process

Review Phase

- code-review-and-quality: Five-axis review, change sizing
- code-simplification: Chesterton's Fence, Rule of 500
- security-and-hardening: OWASP Top 10, auth patterns
- performance-optimization: Core Web Vitals, profiling

Ship Phase

- git-workflow-and-versioning: Trunk-based, atomic commits
- ci-cd-and-automation: Shift Left, feature flags
- deprecation-and-migration: Code-as-liability, migration patterns
- documentation-and-adrs: Architecture Decision Records
- observability-and-instrumentation: Structured logging, RED metrics
- shipping-and-launch: Pre-launch checklists, rollback plans

8. Troubleshooting

Problem: Agent ignores skills

The model may not follow AGENTS.md instructions. Try adding explicit instructions like "Follow the spec-driven-development skill before coding" at the start of your message.

Problem: Skills not found

Verify the skills/ directory exists in your project root. The path in AGENTS.md must match the actual location. Run: dir skills\

Problem: Agent skips the spec phase

Free models sometimes jump to implementation. Add this to your prompt: "DO NOT write code until you have written a spec and I have approved it. Follow spec-driven-development skill exactly."

Problem: Too many tokens / slow responses

Skills load progressively. If responses are slow, invoke only the specific skill you need: "Use the test-driven-development skill only for this task."

Problem: Agent says "I will add tests later"

This is the most common rationalization. Respond with: "No. Follow the Beyonce Rule from the test-driven-development skill. Write the test first."

9. Best Practices

1. Start small. Use one skill at first (e.g., test-driven-development) to get comfortable.
2. Be explicit. Tell the agent which skill to use when starting out.
3. Verify outputs. Check that tests actually pass, not just that the agent claims they do.
4. Review specs. Before implementation, read the spec the agent writes and approve it.
5. Commit often. The incremental-implementation skill commits after each small slice.
6. Use /build auto only after you trust the workflow. It removes human steps between tasks.
7. Read the skill files. Each SKILL.md is a readable workflow - understanding them helps you guide the agent.
8. Report issues. Free models may not follow all instructions perfectly. Adjust prompts as needed.

10. Example Session

Here is a complete example of using agent skills in a session:

You (Message 1)

```
I want to build a task management app with
user auth, tasks, and categories.
Follow the spec-driven-development skill.
Interview me one question at a time.
```

The agent will invoke the interview-me skill and ask you structured questions one at a time until it has enough information to write a PRD.

You (Message 2 - after spec is approved)

```
Good, the spec looks complete.
Now use planning-and-task-breakdown to
decompose this into tasks.
```

The agent produces a task list with acceptance criteria and dependency ordering.

You (Message 3 - start building)

```
Start with the user registration task.
Use test-driven-development.
Write tests first, then implement.
```

The agent follows red-green-refactor: writes a failing test, makes it pass, then refactors. Each step is verified before moving on.